CIRA — Hardware-Constrained Execution Addendum (HCEA v1.0)

Compute, memory and disk policy for Chapters 5–19 of the CIRA Implementation Bible

15 September 2026

Contents

0. Status and scope of this document **2**
0.1 How to use this document with an LLM . . . 2

1. Target machine **3**
1.1 Stated specification . . . 3
1.2 Working budget derived from that specification . . . 3

2. Standing rules (apply to every chapter from 5 onward) **4**

3. Dataset facts and the on-disk contract **5**
3.1 What is on disk (as provided) . . . 5
3.2 Known structural facts (r4.2) . . . 5
3.3 Timestamp parsing (a real performance trap) . . . 5
3.4 Canonical processed layout (created by Chapter 5 Stage 0) . . . 6

4. Execution profiles **6**

5. Chapter 5 — Feature Engineering (the heavy chapter) **7**
5.1 The shape of the problem . . . 7
5.2 Stage 0 — Parquet conversion (run once) . . . 7
5.3 Stage 1 — Domain aggregation to user-day . . . 8
5.4 Stages 2–4 — in-memory (no hardware concern) . . . 9
5.5 Chapter 5 acceptance additions . . . 9
5.6 One optional addition, flagged honestly . . . 9

6. Chapter 6 — Baselines **9**
6.1 LOF — mandatory deviation . . . 10
6.2 LSTM autoencoder — windowing adjustment . . . 10
6.3 Uniform interface . . . 10

7. Chapter 7 — TabNet **11**
7.1 Reality check . . . 11
7.2 Mandatory settings on this machine . . . 11
7.3 Class imbalance — adjustment to the Bible . . . 11
7.4 Pretraining . . . 12
7.5 Split strategy (unchanged, restated because it interacts with profiles) . . . 12
7.6 Budget . . . 12

8. Chapter 8 — Anomaly scoring and serving **12**

9. Chapter 9 — CRI **12**

10. Chapter 10 — MITRE ATT&CK **12**

11. Chapter 11 — Explainability (TabNet masks + SHAP) **13**
11.1 TabNet masks — cheap 13
11.2 SHAP KernelExplainer — the real cost, and the adjustment 13

12. Chapter 12 — Alert correlation and persistence **13**

13. Chapters 13–14 — FastAPI and React **14**

14. Chapters 15–16 — Validation, evaluation and ablations **14**

15. Chapters 17–18 — Production extensions **14**

16. Windows and thermal operating notes **15**

17. New configuration variables **15**

18. Deviation register **15**

19. Open items requiring confirmation **16**

0. Status and scope of this document

Document type: Execution addendum. Not an architecture document.

Authority: This document is **subordinate** to `CIRA_FINAL_ARCHITECTURE.md` and `CIRA_IMPLEMENTATION_BIBLE.md`. It changes **no** technology choice, **no** module boundary, **no** architectural rule and **no** phase ordering. It only constrains *how* each chapter is executed on the specific machine described in §1.

Where this document adjusts something the Bible states (for example, SMOTE vs class-weighted loss, or running LOF on the full matrix), that adjustment is recorded explicitly in the **Deviation Register** in §18 with a justification and a status. No adjustment in this document is silent.

Project status at the time of writing (stated by the developer, not verified by inspection):

Chapter	Title	Status
1	Repository Foundation	IMPLEMENTED
2	Database + FastAPI Skeleton	IMPLEMENTED
3	Dataset Acquisition & Ingestion	IMPLEMENTED
4	Preprocessing & Normalization	IMPLEMENTED
5	Feature Engineering	**NEXT — not started**
6–19	—	PLANNED
Production extensions (Kafka, Redis, Celery, SSE, OpenSearch, Prometheus, Grafana, Kubernetes)	—	NOT IMPLEMENTED

0.1 How to use this document with an LLM

Upload this PDF **together with** `CIRA_FINAL_ARCHITECTURE.md` and `CIRA_IMPLEMENTATION_BIBLE.md`, then use this preamble:

You are implementing CIRA. `CIRA_FINAL_ARCHITECTURE.md` is the architectural source of truth. `CIRA_IMPLEMENTATION_BIBLE.md` is the sequencing source of truth. The Hardware-Constrained Execution Addendum (HCEA) is the execution-budget source of truth: every piece of code you produce must satisfy its standing rules in §2 and the chapter-specific budget for the chapter I name. Do not exceed the memory, disk or thread budgets. Do not introduce a technology that is not already in the Bible's tech stack table without flagging it as an ADDITION REQUIRING APPROVAL. Do not claim anything is implemented that you have not seen run. If something is undefined, say "Not currently defined in the project specification."

---

1. Target machine

1.1 Stated specification

Component	Value
CPU	Intel Core i5-13450HX (hybrid: performance + efficiency cores, mobile HX class)
RAM	32 GB
GPU	NVIDIA laptop GPU, 6 GB VRAM (stated as "3050 6 GB" — **confirm exact model**)
Storage	512 GB NVMe SSD (C:), plus a second volume (D:)
Free space	~78 GB (**confirm which volume**)
OS	Windows
Repository	`C:\...\cira-insider-risk`
Dataset	`D:\CIRA_dataset`

1.2 Working budget derived from that specification

These are the numbers every later chapter is designed against. They are deliberately conservative: the goal is a machine that finishes a job overnight without thermal throttling or an out-of-memory kill, not a machine that runs at 100% and dies at hour six of a seven-hour job.

Resource	Hard ceiling	Target working set	Reserved
Process RAM (any single Python job)	20 GB	8–12 GB	8 GB for OS, browser, IDE, Docker
Docker Desktop / WSL2 RAM	8 GB	4–6 GB	capped in `.wslconfig`
GPU VRAM	5.0 GB usable of 6 GB	< 3 GB	display + headroom
CPU threads for batch jobs	8	6	leave 4+ threads for OS responsiveness
Disk on D: for processed artifacts	25 GB	10–15 GB	—
Disk on C: for models/checkpoints/Postgres	20 GB	8 GB	—
Single-job wall-clock before checkpoint	45 min	15 min	—

Thermal rule. No job is allowed to hold all cores at 100% for more than ~45 minutes without reaching a checkpoint. Long work is decomposed into resumable stages (§2, R7). This is the single

most important protection against a laptop that "fries itself": not lower utilisation, but *interruptible* utilisation.

---

2. Standing rules (apply to every chapter from 5 onward)

R1 — Never load a raw CERT domain file fully into memory. `http.csv` dominates CERT r4.2 in both rows and bytes. Every read of a raw file is chunked (`pandas.read_csv(..., chunksize=...)`) or columnar-batched. There is no exception for "just checking something" — use `nrows=100_000` for inspection.

R2 — Raw CSV is read at most twice: once to profile, once to convert. Chapter 5 Stage 0 converts each normalized domain stream to Parquet (zstd, partitioned by month). Every subsequent read in every subsequent chapter reads Parquet, never CSV. Parquet is columnar, typed and roughly 5–10x cheaper to scan.

R3 — Aggregate early, aggregate hard. CERT r4.2 is ~32.8M events but only ~1,000 users over ~17 months. The user-day feature matrix is therefore on the order of **3×10^5 to 5×10^5 rows**, not 10^7. Every expensive operation (models, SHAP, LOF, evaluation) runs on the *aggregated* matrix. The only code that ever touches 10^7 rows is the Chapter 5 streaming aggregation pass.

R4 — Fix dtypes explicitly. `int32`/`float32`/`category` everywhere. Never let pandas infer `int64`/`float64`/`object` on a multi-million-row frame. `object`-dtype string columns are the main cause of surprise memory blowups.

R5 — Datasets stay on D:, repository stays on C:. No dataset file is ever copied into the repository. Access is via configuration (`CERT_RAW_DIR`, `CERT_PROCESSED_DIR`), never a hard-coded path. `datasets/raw/` and `datasets/processed/` inside the repo remain **empty and gitignored**, or become junction points; the canonical location is `D:\CIRA_dataset`.

R6 — Thread caps are explicit, not default. Set before any BLAS/sklearn/torch import: `OMP_NUM_THREADS=6`, `MKL_NUM_THREADS=6`, `n_jobs=6`, `torch.set_num_threads(6)`. Library defaults will grab every logical core and leave the machine unusable.

R7 — Every long stage is checkpointed and resumable. Each stage writes its output atomically (write to `.tmp`, then rename) and skips itself on re-run if its output exists and its input hash is unchanged. A crash at 80% must cost 20% of the work, not 100%.

R8 — Every heavy script logs a resource line. Wall-clock, peak RSS, rows in, rows out, output path, config hash — appended to `experiments/runlog.jsonl`. This log is also the honest evidence trail required by Bible Chapter 19 and Architecture §37/§38.

R9 — Work up the profile ladder, never straight to full scale. Every new script is proven on `dev`, then `mid`, then `full` (§4). Debug at `dev`; `full` is run once, deliberately, usually overnight.

R10 — Results reported in the report/viva come only from a `full` (or explicitly labelled `mid`) run. Numbers from `dev` are development artefacts and are never reported. This preserves Architecture §47 rule 23 (no fabricated results).

R11 — GPU is optional everywhere except the LSTM autoencoder. With ~200 tabular features, CPU training is competitive. If CUDA is unavailable or VRAM is tight, every model in this project must still train on CPU. No code path may require the GPU.

R12 — PostgreSQL stores decisions and lineage, not the whole corpus. See §12. The 32.8M raw events are not bulk-loaded into Postgres for the FYP.

---

3. Dataset facts and the on-disk contract

3.1 What is on disk (as provided)

```
D:\CIRA_dataset
|   answers.tar.bz2          <- ground-truth archive (keep)
|   r4.2.tar                 <- REDUNDANT once extracted
|   r4.2.tar.bz2             <- REDUNDANT once extracted (keep ONE, not both)
\---datasets
    +---ground_truth\cert_r4.2\   (r4.2-1 = 30 insiders, r4.2-2 = 30,
    |                              r4.2-3 = 10; plus r2/r3/r5/r6 answers,
    |                              scenarios.txt, insiders.csv)
    \---raw\cert_r4.2\
            device.csv  email.csv  file.csv  http.csv
            logon.csv   psychometric.csv
            LDAP\  2009-12.csv ... 2011-05.csv   (18 monthly snapshots)
```

Immediate action — reclaim disk. After verifying the extraction is complete, delete **one** of `r4.2.tar` / `r4.2.tar.bz2`. Keeping the extracted tree plus two archives of the same tree is pure waste on a volume that is the binding constraint of this project. Keep `answers.tar.bz2` (small).

Scope note. The ground-truth folder contains answers for r2, r3.x, r5.x and r6.x. Only `insiders.csv` (filtered to r4.2) and `r4.2-1/`, `r4.2-2/`, `r4.2-3/` are in scope. Everything else is ignored — not deleted, just never referenced.

3.2 Known structural facts (r4.2)

- ~1,000 users; ~17 months of activity (LDAP snapshots span 2009-12 to 2011-05).
- ~32.77M events total; 7,323 labelled malicious events (figures as stated in the Bible, Chapter 3).
- 70 malicious insiders: 30 in scenario 1, 30 in scenario 2, 10 in scenario 3 — consistent with the per-user answer files present on disk.
- `http.csv` is by far the largest file by rows and bytes; `psychometric.csv` is ~1,000 rows.

Do not plan against assumed byte sizes. Measure once, record the result in the repo, and plan from measured numbers:

```powershell
Get-ChildItem "D:\CIRA_dataset\datasets\raw\cert_r4.2" -Recurse -File |
  Select-Object FullName,
    @{n='MB';e={[math]::Round($_.Length/1MB,1)}} |
  Sort-Object MB -Descending | Format-Table -AutoSize
```

Save the output to `experiments/dataset_inventory.txt` and commit it. Chapter 5's budget is then derived from real numbers rather than from a guess.

3.3 Timestamp parsing (a real performance trap)

CERT timestamps are `MM/DD/YYYY HH:MM:SS`. Calling `pd.to_datetime` without a format on tens of millions of strings is one of the slowest operations in this pipeline.

Two rules:

1. Always pass `format="%m/%d/%Y %H:%M:%S"`.
2. For the day key and hour bucket, prefer string slicing over datetime parsing:

```python
day  = s.str.slice(0, 10)            # "01/02/2010"  -> group key
hour = s.str.slice(11, 13).astype("int8")
```

Full datetime parsing is only needed where true time arithmetic is required (session gaps, inter-event deltas). Everywhere else, slice.

3.4 Canonical processed layout (created by Chapter 5 Stage 0)

```
D:\CIRA_dataset\datasets\processed\
    events\                      (Parquet, zstd, partitioned by month)
        source_type=logon\month=2010-01\part-0.parquet
        source_type=device\...
        source_type=file\...
        source_type=email\...
        source_type=http\...
    context\
        ldap_user_month.parquet         (user x month role/dept/supervisor)
        psychometric.parquet
    labels\
        insider_user_days.parquet       (user, date, scenario, is_malicious)
    features\
        user_day_dev.parquet
        user_day_mid.parquet
        user_day_full.parquet
        feature_schema.json
    _runlog\
```

Labels live in a **separate directory tree** from features and are joined only inside the evaluation harness — the Architecture §9 anti-leakage rule, enforced by layout and not merely by convention.

---

4. Execution profiles

A single environment variable, `CIRA_PROFILE`, selects the scale of every run. It is read once in `app/core/config.py` and threaded through the pipeline.

Profile	Users	Time span	Purpose	Target runtime (Ch5 full pass)
`dev`	50 (incl. 10 insiders)	3 months	write and debug code	seconds to ~2 min
`mid`	250 (all 70 insiders + 180 benign)	full 17 months	tuning, ablations, iteration	minutes to ~30 min
`full`	all ~1,000	full 17 months	final reported results	run once, overnight, checkpointed

Sampling rule. Profiles subsample **by user**, never by row and never by day. A user is either entirely in or entirely out. This keeps per-user history intact (required for historical-baseline features), keeps the train/test split strategy in Chapter 7 valid, and avoids the silent leakage that row-level sampling would create.

Honesty rule. The profile used for any number is recorded next to that number in `experiments/results/`. A `mid` result is reported as a `mid` result.

Why this matters more than any other adjustment in this document: it converts the question "will my laptop survive Chapter 5?" into "how long does the one final `full` run take?" — and that run happens once per pipeline change, not once per code edit.

---

5. Chapter 5 — Feature Engineering (the heavy chapter)

The Bible's Chapter 5 specification is unchanged: ten modules (`authentication`, `file_activity`, `email`, `usb`, `network`, `application`, `temporal`, `historical_baseline`, `peer_group`, `pipeline`), one feature vector per user-day, `feature_schema.json` written, no label leakage. What follows is *how* those modules run on this machine.

5.1 The shape of the problem

```
~32,800,000 raw events   (10^7 scale — streamed, never resident)
            |
            |  Stage 0: normalize + convert to Parquet   (one pass per domain)
            ▼
~32,800,000 typed Parquet rows  (columnar, ~5-10x cheaper to scan)
            |
            |  Stage 1: per-domain chunked aggregation to (user, day)
            ▼
~300,000-500,000 user-day rows  (10^5 scale — fits in RAM comfortably)
            |
            |  Stages 2-4: joins, temporal, baselines, peers  (all in-memory)
            ▼
  feature matrix  [N x ~150-250]  float32  ≈ 200-500 MB
```

The entire compute risk of this project lives in Stage 0 and Stage 1. Everything after Stage 1 operates on a few hundred megabytes and is not a hardware concern.

5.2 Stage 0 — Parquet conversion (run once)

One chunked pass per domain file. Per chunk: apply the Chapter 4 normalization and validation functions, cast dtypes, derive `day` and `hour` by string slicing, drop columns that will never be used, write a Parquet part file.

```
DTYPES = {"id": "string", "user": "category", "pc": "category"}
READER = dict(chunksize=500_000, dtype=DTYPES,
              usecols=DOMAIN_COLUMNS[domain])

for i, chunk in enumerate(pd.read_csv(src, **READER)):
    chunk = normalize(chunk)          # Chapter 4 code, unchanged
    chunk = validate(chunk)           # Chapter 4 code, unchanged
    write_parquet_part(chunk, out_dir, part=i)   # zstd, atomic rename
```

Budget and policy:

- `chunksize` 500,000 for `http.csv`; 1,000,000 for the smaller domains. If peak RSS exceeds 8 GB, halve it. Do not "tune" this upward for speed.
- **Drop the raw URL string from `http`.** Retain only: `user`, `day`, `hour`, `pc`, a `domain` (host) column as `category`, and derived boolean flags (cloud-storage host, job-search host, hacking/keylogger host, upload-capable host). The full URL text is the single largest column in the dataset and contributes nothing to the feature set defined in Architecture §10. Retaining it would roughly double both disk and scan cost for zero modelling value. *Traceability is preserved* by keeping the source row id, per Chapter 3.
- Host classification uses a small, committed, human-reviewed list in `app/feature_engineering/network_dom` It is an explicit expert list, like the MITRE mapping table in Chapter 10 — reviewable, not invented at runtime.
- Checkpoint per part file. Re-running skips completed parts (R7).

Expected outcome: the processed Parquet tree should be **substantially smaller** than the raw CSV tree. Record the actual before/after sizes in `runlog.jsonl`.

5.3 Stage 1 — Domain aggregation to user-day

Each domain module exposes the same shape:

```
def aggregate(parquet_glob, profile) -> pd.DataFrame:
    """Return a frame indexed by (user_id, date) with this domain's
    columns only. Must be chunk-safe: partial results combine by sum."""
```

Two categories of aggregate, handled differently:

(a) Additive aggregates — counts, sums, min/max. Aggregate per chunk, then combine partials with `groupby(level=[0,1]).sum()`. Trivially chunk-safe.

(b) Distinct counts — unique PCs per user-day, unique destination hosts per user-day. `sum()` is *wrong* here. Correct pattern: per chunk, emit a deduplicated `(user, day, item)` triple frame with integer-coded columns; concatenate; deduplicate globally once; then count. With integer codes this intermediate stays in the low hundreds of MB even for `http`.

```
trip = chunk[["user", "day", "host"]].drop_duplicates()
# ... accumulate, then once at the end:
uniq = pd.concat(parts).drop_duplicates()
n_unique = uniq.groupby(["user", "day"]).size()
```

Do not compute distinct counts with `nunique()` inside a chunked loop and add the results — that silently over-counts and would be an invented number. This is exactly the kind of error that produces plausible-looking but wrong evaluation results.

Per-domain notes:

Domain	Cost	Notes
`logon`	low	on/off-hours split, weekend flag, distinct PCs, first/last activity hour
`device`	low	USB connect/disconnect counts, off-hours USB, distinct PCs
`file`	low	counts by extension class, to-removable-media flag, decoy-file access
`email`	medium	sent/received, attachment count and size, internal vs external recipient ratio, distinct external domains. **No NLP, no message-content features** (Architecture §10.3, §47 rule 17)
`http`	**high**	count, distinct hosts, and the boolean host-class flags only. This is the only genuinely expensive aggregation in the project
LDAP	trivial	18 small CSVs; produces the user x month role/department/supervisor context table
psychometric	trivial	~1,000 rows; OCEAN scores as static per-user context

Run each domain as a **separate resumable stage** writing its own intermediate Parquet. The or-

chestration in `pipeline.py` joins the six intermediates on `(user_id, date)`. If `http` fails at hour two, the other five are already done.

5.4 Stages 2–4 — in-memory (no hardware concern)

Everything below operates on the ~10^5-row matrix.

- **Calendar spine.** Build the full `(user, date)` grid for each user's active window and left-join the domain aggregates so "no activity" is explicit, then apply the documented missing-value policy from Chapter 4 (absent is not the same as zero for every column — record the per-column decision in `feature_schema.json`).
- **Temporal features.** Hour-of-day statistics, day-of-week, weekend and off-hours indicators, rolling activity counts.
- **Historical baseline.** Per-user rolling z-scores over a trailing 30-day window. Two non-negotiables: `groupby("user_id") + .rolling(30) + .shift(1)` so the current day never contributes to its own baseline, and a `min_periods` threshold (suggested 7) below which the deviation column is null rather than fabricated. Record both choices in `feature_schema.json` (Bible Ch5 step 8 requires the statistical method be recorded, not silently invented).
- **Peer group.** Join the LDAP user x month table, compute department-level medians and per-user deviation from them. CERT genuinely supplies role and department, so this is supported; nothing is invented (Architecture §10.9).
- **Output.** `float32` matrix + `(user_id, date)` index columns + `feature_schema.json` (name, dtype, source domain, null policy, baseline method per column).

5.5 Chapter 5 acceptance additions

In addition to the Bible's acceptance checklist:

- ☐ `dev` profile end-to-end in under ~2 minutes.
- ☐ Peak RSS of the full pipeline logged and under 12 GB.
- ☐ `http` aggregation is resumable — verified by killing it mid-run and re-running.
- ☐ Distinct-count features validated against a brute-force computation on the `dev` profile (guards the chunk-safety bug described in §5.3).
- ☐ `feature_schema.json` records, for every column: dtype, source domain, null policy, and (where applicable) baseline window and `min_periods`.
- ☐ No label column, and no column derived from one, exists in the matrix.

5.6 One optional addition, flagged honestly

DuckDB is not currently part of the project specification. It would make Stage 1 a handful of SQL statements executed out-of-core over the Parquet tree, with better memory behaviour and much less hand-written chunking code, at the cost of one new dependency (a single wheel, no server, ~50 MB).

- **Why it would help:** removes the entire class of chunk-safety bugs in §5.3.
- **What it costs:** one dependency outside the Bible's tech-stack table, and a second idiom (SQL) in the feature-engineering layer.
- **Recommendation:** default to **pandas chunked**, which is already in the mandated stack and is sufficient at this data scale. Adopt DuckDB only if the `full http` aggregation proves unacceptably slow *after* measurement — and if adopted, record it as an explicit approved addition, not a silent one.

---

6. Chapter 6 — Baselines

All baselines consume the same `[N x F]` float32 matrix. `N ≈ 3–5 x 10⁵`, `F ≈ 150–250`.

Baseline	Feasibility	Required adjustment
Rule-based	trivial	none
Isolation Forest	fine	`n_estimators=100`, `max_samples=256`, `n_jobs=6`, `random_state` fixed. Sub-sampling is the algorithm's own design, not a compromise
LOF	**not feasible as specified**	see §6.1
LSTM Autoencoder	feasible with care	see §6.2
XGBoost / CatBoost	fine	`tree_method="hist"`, `n_jobs=6`, or `device="cuda"` if VRAM allows

6.1 LOF — mandatory deviation

`LocalOutlierFactor` needs a k-nearest-neighbour search. In ~200 dimensions, tree indices degenerate and scikit-learn falls back to brute force, which is quadratic in N. At N ≈ 400,000 that is on the order of 10^{11} distance computations — hours to days, with a large memory footprint. This is the one place where the Bible's specification cannot be executed literally on this machine (or, realistically, on a much larger one).

Adjustment (recorded in §18, D-2):

1. Reduce dimensionality with PCA fitted on the training split only (target: ~20 components, report retained variance).
2. Fit `LocalOutlierFactor(n_neighbors=20, novelty=True, n_jobs=6)` on a **stratified training subsample of ≤ 50,000 user-days**.
3. Score the full test split in chunks of 50,000 rows via `score_samples`.
4. Report the subsample size, the PCA dimensionality and the retained variance alongside every LOF metric. LOF is a reference baseline, and a documented, reproducible approximation is scientifically honest; a fabricated or omitted one is not.

6.2 LSTM autoencoder — windowing adjustment

Materialising every 30-day window for every user at full feature width is the failure mode here: ~400,000 windows × 30 days × 200 features × 4 bytes ≈ 10 GB, before the model allocates anything.

Adjustment (recorded in §18, D-3):

- Restrict the autoencoder to a documented **core numeric subset** (~30–40 columns: counts and volumes, not one-hot context).
- Window length 30 days, **stride 7** (not 1).
- Generate windows lazily in a `torch.utils.data.Dataset` from a compact `(user, day)`-indexed array; never build the full tensor.
- `batch_size=128`, hidden size 64, 1–2 layers, `num_workers=0` (Windows), AMP on. VRAM footprint under ~1.5 GB.
- Fixed seed; checkpoint every epoch.

6.3 Uniform interface

The Bible's `fit(X_train)` / `score(X_test) -> anomaly_score` contract is unchanged and now also covers LOF's PCA+subsample wrapper. Each baseline additionally persists `{model, config, profile, seed, wall_clock, peak_rss}` so Chapter 16 can report comparable numbers with provenance.

---

7. Chapter 7 — TabNet

7.1 Reality check

TabNet on ~400,000 rows × ~200 features is a **small** deep-learning job. 6 GB VRAM is not the binding constraint; data loading and CPU-side batching are. This chapter is comfortably within budget provided the settings below are used.

7.2 Mandatory settings on this machine

```
TabNetClassifier(
    n_d=16, n_a=16,          # start here; 8 if underfitting is not the problem
    n_steps=4,               # 3-5; each step costs time and VRAM linearly
    gamma=1.3,
    n_independent=2, n_shared=2,
    mask_type="sparsemax",   # entmax only if sparsemax underperforms
    optimizer_params=dict(lr=2e-2),
    scheduler_params=dict(step_size=10, gamma=0.9),
    seed=42, verbose=1, device_name="auto",
)

clf.fit(
    X_train, y_train, eval_set=[(X_valid, y_valid)],
    eval_metric=["auc"],           # PR-oriented metric added in Ch16
    max_epochs=100, patience=15,
    batch_size=4096,
    virtual_batch_size=256,        # ghost batch norm; keep <= 512
    num_workers=0,                 # REQUIRED on Windows
    drop_last=False,
    weights=1,                     # see 7.3
)
```

- `num_workers=0` is not a preference. On Windows, worker processes are spawned, not forked, which re-imports the module and can duplicate a large training array per worker — a fast route to exhausting 32 GB.
- `batch_size=4096` / `virtual_batch_size=256` keeps VRAM in the hundreds of MB. If VRAM errors appear, halve `batch_size` before touching `n_d`/`n_a`/`n_steps`.
- If CUDA is unavailable, `device_name="auto"` falls back to CPU. With `torch.set_num_threads(6)` the CPU path on this CPU is acceptable. Record which device each run used (R8).

7.3 Class imbalance — adjustment to the Bible

The Bible (Chapter 7 step 3) allows "resampling (e.g. SMOTE) **or** class-weighted loss", and requires recording which was used.

Selection: class-weighted loss. SMOTE over a ~400,000 × 200 matrix synthesises minority rows by interpolating between neighbours — expensive here (another kNN in high dimensions, §6.1) and semantically dubious: an interpolated "half-way insider day" is not a behaviour any user exhibited, and it corrupts the per-user temporal structure the rest of the pipeline depends on.

Implementation: `pytorch-tabnet`'s `weights=1` (automatic inverse-frequency sampling weights) or an explicit `class_weight` in the loss. Record the chosen option, the effective weight, and the positive rate in the model registry entry. If time permits, run the alternative once at `mid` profile and report both — a one-line ablation that costs little and strengthens the evaluation chapter.

7.4 Pretraining

`TabNetPretrainer` (self-supervised) is **optional** and time-boxed: attempt at `mid`, maximum 20 epochs, and keep it only if it measurably improves validation PR-AUC. It is not required for a complete FYP and must not become a rabbit hole.

7.5 Split strategy (unchanged, restated because it interacts with profiles)

Leakage-safe splitting is by **user** or by **time**, never random row mixing. With user-based profiles (§4), the recommended scheme is: hold out a disjoint set of users *including a proportional share of the 70 insiders*, and additionally report a time-based split (train on the first ~13 months, test on the last ~4). Both splits are documented in `experiments/`.

7.6 Budget

Item	Budget
Peak RSS	< 10 GB
Peak VRAM	< 3 GB
Wall-clock, `mid`	minutes
Wall-clock, `full`	tens of minutes; checkpoint per epoch
Hyperparameter search	manual, ≤ 12 configurations, logged. **No AutoML, no unattended overnight grid search**

---

8. Chapter 8 — Anomaly scoring and serving

- Load the model **once** at FastAPI startup (lifespan handler), never per request.
- **Serve on CPU.** The model is small, request volume in this project is low, and keeping the API off the GPU leaves VRAM free for training and SHAP work. Explicitly set `device_name="cpu"` for the serving instance.
- Batch scoring runs as a standalone script writing scores to Parquet, then a bounded load into Postgres (§12) — never row-by-row inserts over an HTTP API.
- The §36 “model unavailable” failure mode is unchanged: explicit error, never a fabricated score.
- Every score carries its model version (§37 lineage).

9. Chapter 9 — CRI

Negligible compute: an arithmetic combination over the user-day matrix. Vectorise it (operate on the whole frame, not a Python loop over rows) and it runs in seconds at `full` scale.

The only hardware-relevant note: the CRI ablation hooks required by Chapter 16 should switch components **by configuration on a single scored matrix**, so the four ablation variants cost four cheap recomputations rather than four full pipeline re-runs.

10. Chapter 10 — MITRE ATT&CK

The `enterprise-attack` STIX bundle is a large JSON document, and `mitreattack-python`’s in-memory object graph is heavy — acceptable in a one-off script, wasteful inside a long-lived API process.

Adjustment (recorded in §18, D-4): run `stix_loader.py` **offline, once**, against a pinned bundle version, and materialise a small local lookup table (`technique_id`, `name`, `tactic`, `short_description`, `bundle_version`) as JSON or a Postgres table. The FastAPI process and the enrichment path read only that table. This satisfies Chapter 10's "reproducible, no runtime network dependency" requirement and removes a multi-hundred-megabyte resident object from the API. Mapping logic in `mapping_rules.py` is unchanged; unmapped stays unmapped.

11. Chapter 11 — Explainability (TabNet masks + SHAP)

11.1 TabNet masks — cheap

`clf.explain(X)` over the full matrix returns an `[N x F]` importance array (~400,000 × 200 × 4 B ≈ 320 MB). Run it in chunks of 50,000 rows and persist only what is needed: the top-k contributing features per row, not the dense matrix.

11.2 SHAP KernelExplainer — the real cost, and the adjustment

`shap.KernelExplainer` cost is roughly `rows_explained × nsamples × background_size` model evaluations. Running it over the full matrix is not feasible and is not required.

Adjustment (recorded in §18, D-5): SHAP is computed on demand for alerts only.

- Background set: `shap.kmeans(X_train, 50)` — **never** the full training set.
- `nsamples=100` (raise to 200 only if explanations are unstable).
- Explain **only** the rows that became alerts, plus the top-k ranked risks for the dashboard. In practice: tens to low hundreds of rows per run, not 400,000.
- Persist SHAP contributions to `AlertReason` so each is computed once.
- Everything else in the explanation — TabNet masks, CRI factors, baseline deviations, MITRE context — is already cheap and available for every row.

This is consistent with the Bible: SHAP is the *corroborating* signal; TabNet masks are the primary model-side explanation. Architecture §36's "SHAP unavailable" degradation path (alert exists, explanation retried later) is what makes on-demand computation architecturally clean rather than a shortcut.

Note: the Bible places the asynchronous SHAP batch path in Celery (Chapter 17, production extension). Until then, the batch path is a **plain script**, run manually. Celery is not introduced early to satisfy this need.

12. Chapter 12 — Alert correlation and persistence

Adjustment (recorded in §18, D-6): bounded event persistence.

Bulk-loading 32.8M events into PostgreSQL would consume roughly 10–15 GB of table plus index space in a Docker volume that lives on the C: drive, slow every migration, and serve no analyst need in the FYP.

Policy:

- `EventLog` persists **only** events linked to a persisted `Alert`, plus a bounded, deterministic sample for demonstration purposes (suggested: all events for ~20 users across the demo window). The sample's selection rule is recorded in configuration.
- `FeatureVector` persists the user-days that produced alerts, plus the same demo sample — not all ~400,000.
- `AnomalyScore` / `RiskScore` persist for the demo window and for all alerting rows.
- The **full** score set stays in Parquet under `datasets/processed/`, referenced by path + row id. Lineage (§37) is preserved end to end: every persisted alert traces Source Event → Normalized

Event → Feature Vector → Model Version → Anomaly Score → CRI Inputs → Risk Score → MITRE Mapping → Alert.
- OpenSearch, whose actual job is large-scale event search, remains a Chapter 18 production extension. The bounded policy above is precisely what makes its absence defensible in the FYP.

Operational notes:

- Use `COPY` / `psycopg` batch insert for the bounded load, not per-row ORM inserts.
- Add indexes **after** the bulk load, not before.
- Docker volumes live on C: by default (WSL2 virtual disk). Monitor free space and cap WSL2 memory (§16).

13. Chapters 13–14 — FastAPI and React

No heavy compute, but three practical constraints on this machine:

- **Never return an unbounded result set.** Every list endpoint is paginated with a hard server-side cap. A "return all risk scores" endpoint would try to serialise hundreds of thousands of rows into JSON and stall both the backend and the browser.
- **Aggregate server-side.** Risk-history and overview charts receive pre-aggregated series (daily or weekly buckets), not raw rows for Recharts to reduce in the browser.
- **Dev-time memory.** Vite dev server, Node, VS Code, Docker Desktop and a Python training job do not coexist peacefully in 32 GB. Do not run frontend development and a `full`-profile training job at the same time.

14. Chapters 15–16 — Validation, evaluation and ablations

- **E2E tests use a tiny fixture**, not the CERT corpus: a handful of synthetic events and a small checkpointed model committed under `tests/fixtures/`. The e2e suite must run in under a minute so it can be run on every change (Bible Chapter 15 step 4).
- **testcontainers PostgreSQL** starts a real container per integration session. Use a session-scoped fixture (one container per run, not per test) — container churn is the main cost here.
- **Bootstrap confidence intervals** resample indices, not data: 1,000 resamples over a few hundred thousand precomputed scores is seconds of work. No excuse to skip §39's statistical rigour.
- **Ablations A–D** (TabNet / +context / +CRI / +CRI+MITRE) reuse one scored matrix and recompute only the layers under test. Only Experiment A/B require retraining; C and D are configuration changes over the same scores. Budget: one `full` training run plus cheap recomputation, not four full runs.
- **Cross-validation**: 3 folds at `full`, 5 at `mid`. Report the number of folds.

15. Chapters 17–18 — Production extensions

Unchanged in status: **NOT IMPLEMENTED, not required for the FYP core.**

If attempted after Chapter 16 is genuinely complete, the memory arithmetic on this machine is unforgiving. Indicative resident footprints: Redpanda ~1–2 GB, Redis ~0.1 GB, a Celery worker ~0.5–1 GB, OpenSearch ~2 GB (JVM heap, single node), Prometheus + Grafana ~0.7 GB, plus PostgreSQL, the backend, the frontend and the OS. Running all of them together with a training job is not viable.

Policy if this phase is reached:

- Bring services up **one at a time**, in a separate `docker-compose.prod.yml` overlay, and demonstrate each individually.

- Set explicit `mem_limit` on every service.
- Never run a production-extension demo and a model training run simultaneously.
- Kubernetes: document only. Do not attempt a local cluster on this hardware.

16. Windows and thermal operating notes

- **Windows Defender exclusion** for `D:\CIRA_dataset` and the repository's virtual environment directory. Real-time scanning of millions of file reads and of every Python import is a large, invisible I/O tax. (User's own security decision — noted, not prescribed.)
- **`.wslconfig`** in `C:\Users\<you>\`:

```
[wsl2]
memory=8GB
processors=6
swap=4GB
```

Without this, WSL2/Docker Desktop will claim up to half of physical RAM by default.

- **Keep the page file system-managed.** With 32 GB and a 20 GB ceiling, a brief spike should page rather than hard-fail.
- **Thermals.** Sustained all-core load on an HX-class mobile CPU will throttle. The mitigation is architectural, not cosmetic: checkpointed stages (R7) mean throttling costs time, never work. A hard flat surface and the "Balanced"/"Best performance" power plan while plugged in are worth more than any software tuning.
- **Long runs**: disable sleep/hibernate for the duration; run heavy stages one at a time, not in parallel shells.
- **Monitor, don't guess.** `psutil` (small dependency, recommended addition) in the runlog gives peak RSS per stage; Task Manager gives GPU/VRAM and CPU temperature trends. Budget violations should be visible in the log, not discovered by a crash.

17. New configuration variables

Extending Architecture §43. These are configuration only — no architectural change.

```
CIRA_PROFILE=dev|mid|full
CERT_RAW_DIR=D:/CIRA_dataset/datasets/raw/cert_r4.2
CERT_GROUND_TRUTH_DIR=D:/CIRA_dataset/datasets/ground_truth/cert_r4.2
CERT_PROCESSED_DIR=D:/CIRA_dataset/datasets/processed
CIRA_MAX_WORKERS=6
CIRA_CHUNK_ROWS=500000
CIRA_MAX_RSS_GB=20
CIRA_DEVICE=auto|cpu|cuda
CIRA_SEED=42
```

All are added to `.env.example` with safe placeholders. `.env` remains uncommitted. Absolute dataset paths are machine-specific and therefore belong in `.env`, never in source.

18. Deviation register

Every place this addendum departs from a literal reading of the Bible, with justification and status. Nothing here is silent; nothing here changes the architecture.

D-1 — Chapter 5 feature pipeline. *Status: MANDATORY.* Specification: build the feature pipeline over CERT r4.2. Adjustment: Parquet conversion first, chunked per-domain aggregation, profile ladder (`dev`/`mid`/`full`). Justification: 32.8M events cannot be held in memory; aggregation collapses the problem to ~10^5 rows, after which nothing downstream is hardware-limited.

D-2 — LOF (Bible Ch6.3, Architecture §13). *Status: MANDATORY.* Specification: LOF on the feature matrix. Adjustment: PCA to ~20 dimensions, fit on a stratified training subsample of <= 50,000 rows with `novelty=True`, score the test split in chunks; report subsample size, PCA dimensionality and retained variance beside every LOF metric. Justification: k-nearest-neighbour search in ~200 dimensions at N ~ 4x10^5 degenerates to brute force and is effectively quadratic — not runnable on this machine or on a much larger one.

D-3 — LSTM autoencoder (Bible Ch6.4). *Status: MANDATORY.* Specification: sequence autoencoder over per-user windows. Adjustment: documented core numeric feature subset, window 30 days with stride 7, lazy window generation in a `Dataset`. Justification: materialising full-width windows costs roughly 10 GB before the model allocates anything.

D-4 — MITRE STIX loading (Bible Ch10.1). *Status: RECOMMENDED.* Specification: load the pinned `enterprise-attack` bundle via `mitreattack-python`. Adjustment: run the loader offline once and materialise a small technique lookup table; the API and enrichment path read only that table. Justification: keeps a heavy in-memory object graph out of a long-lived process while remaining pinned, offline and reproducible exactly as the chapter requires.

D-5 — SHAP (Bible Ch11.2). *Status: MANDATORY.* Specification: `shap.KernelExplainer` as the corroborating explanation signal. Adjustment: computed on demand for alert rows and dashboard top-k only; background set `shap.kmeans(X_train, 50)`; `nsamples=100`; results cached to `AlertReason`. Justification: KernelExplainer over the full matrix is infeasible and unnecessary — TabNet masks already cover every row, and §36 defines the deferred-explanation path.

D-6 — Event persistence (Bible Ch12.3, Architecture §20/§37). *Status: RECOMMENDED.* Specification: persist `EventLog`, `FeatureVector`, anomaly and risk scores. Adjustment: bounded persistence — alert-linked rows plus a deterministic demo sample; the full set stays in Parquet, referenced by path and row id. Justification: full-corpus persistence costs roughly 10–15 GB on the constrained volume for no analyst benefit in the FYP; the §37 lineage chain is preserved intact for every persisted alert.

D-7 — Class imbalance (Bible Ch7.3). *Status: WITHIN SPECIFICATION.* Specification: "resampling (e.g. SMOTE) **or** class-weighted loss", with the choice recorded. Adjustment: class-weighted loss selected, recorded in the model registry with the effective weight and positive rate. Justification: explicitly permitted by the specification. SMOTE additionally requires a high-dimensional neighbour search (see D-2) and interpolates synthetic user-days that distort the per-user temporal structure the rest of the pipeline depends on.

D-8 — HTTP URL retention (Architecture §10.5). *Status: RECOMMENDED.* Specification: network features including destination information. Adjustment: raw URL strings dropped at Stage 0; host plus expert host-class flags retained. Justification: the URL column is the largest in the corpus and contributes nothing to the §10 feature set; the source row id preserves traceability to the original record.

D-9 — DuckDB. *Status: REQUIRES APPROVAL — NOT ADOPTED.* Not currently defined in the project specification. Proposed only as an optional aid to the chunk-safety problem in §5.3. Default remains pandas chunked processing.

D-10 — psutil. *Status: REQUIRES APPROVAL — RECOMMENDED.* Small dependency supporting the R8 resource-logging evidence trail. Not currently in the Bible's tech-stack table.

19. Open items requiring confirmation

1. **Exact GPU model and VRAM.** The stated "3056 GB" is read as an RTX 3050 laptop GPU with 6 GB. Confirm with `nvidia-smi`; the VRAM budgets in §7 depend on it.
2. **Which volume has ~78 GB free** — C: or D:? Determines whether the Docker/Postgres budget (§12) or the processed-artifacts budget (§3.4) is the tighter constraint.
3. **Measured raw file sizes** (§3.2 command). Every disk estimate here should be replaced by measured values.

4. **Chapter 4's actual output format and location.** If Chapter 4 currently writes CSV, its sink changes to Parquet as described in §5.2 — a change to a sink, not to logic. Whether the Chapter 4 output already lives under `D:\CIRA_dataset\datasets\processed` needs confirming before Chapter 5 Stage 0 is written.
5. **TWOS status.** Still tracked honestly as PLANNED / IN PROGRESS unless the dataset is actually in hand. Nothing in this addendum assumes TWOS.
6. **D-9 and D-10 approvals** (DuckDB, psutil).

End of HCEA v1.0. This document is an execution budget, not an architecture. Where it appears to conflict with `CIRA_FINAL_ARCHITECTURE.md`, the architecture document wins and this document is wrong and should be corrected.